Fix Backend API Database Issues in Script Upload Pipeline

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Fix all database-related bugs in the script upload pipeline that cause silent failures, data loss, and broken status tracking.

Architecture: Six verified bugs across AsyncUploadController, Script model, ScriptVersion model, and fileIntegrity utility. Fixes target model definitions, controller logic, and a new DB migration for hash uniqueness. Each task is independent and produces a working, testable change.

> **Migration ordering:** Task 5 (MD5→SHA-256 rehash) **must run before** Task 4 (UNIQUE constraint). If duplicate MD5 hashes exist in the database, adding the UNIQUE constraint will fail. Task 5 recomputes all hashes as SHA-256, eliminating duplicates caused by MD5 collisions, so the constraint can be safely added afterward.
>
> **Deployment sequence (no-skew):** To prevent a window where old nodes write MD5 hashes while new nodes write SHA-256 (which would bypass deduplication), deploy in this order: 1. **Deploy code changes** (Tasks 1-3, 5 code, 6) to **all instances** — the new code writes SHA-256 but still reads both MD5 and SHA-256 via the deprecated aliases 2. **Run Task 5 migration** (rehash existing MD5→SHA-256) — now all hashes in DB are SHA-256 3. **Run Task 4 migration** (audit duplicates, reassign dependents, add UNIQUE constraint) 4. **Verify:** `SELECT COUNT(*) FROM scripts WHERE length(file_hash) = 32;` should return 0
>
> In a single-instance deployment (Docker Compose), steps 1-3 can be done in sequence during a single maintenance window. For multi-instance deployments, step 1 must complete on all instances before step 2.

Tech Stack: TypeScript, Sequelize 6, PostgreSQL 15, Express

File Structure

Action	File	Responsibility
Modify	`src/backend/src/models/Script.ts`	Add `hasMany(ScriptVersion)` association, mark `file_hash` index unique
Modify	`src/backend/src/controllers/AsyncUploadController.ts`	Fix `Script.create` (missing content, ghost uploadId), fix `changes` → `changelog`, fix `getUploadStatus` query
Modify	`src/backend/src/controllers/script/export.ts`	Move hash check inside transaction
Modify	`src/backend/src/utils/fileIntegrity.ts`	Upgrade MD5 → SHA-256
Create	`src/db/migrations/upgrade_file_hash_sha256.sql`	Rehash MD5→SHA-256 (run first)
Create	`src/db/migrations/add_unique_file_hash.sql`	Add UNIQUE constraint on `file_hash` (run after rehash)
Create	`src/backend/src/models/__tests__/Script.associations.test.ts`	Association tests
Create	`src/backend/src/controllers/__tests__/AsyncUploadController.test.ts`	Unit tests for upload pipeline fixes

Task 1: Add Missing `hasMany(ScriptVersion)` Association to Script Model

Files: - Modify: `src/backend/src/models/Script.ts:100-110` (associate method)

The `AsyncUploadController.getUploadStatus` (line 194) does `include: [{ model: ScriptVersion, as: 'versions' }]` but `Script` has no `hasMany(ScriptVersion)` association. This causes a Sequelize eager-loading error.

- [ ] **Step 1: Write the failing test**

Create file `src/backend/src/models/__tests__/Script.associations.test.ts`:

```
import Script from '../Script';
import ScriptVersion from '../ScriptVersion';
import { sequelize } from '../../database/connection';

// Initialize models (mimics src/models/index.ts)
Script.initialize(sequelize);
ScriptVersion.initialize(sequelize);
Script.associate();
ScriptVersion.associate();

describe('Script model associations', () => {
  it('should have a hasMany association with ScriptVersion as "versions"', () => {
    const associations = Script.associations;
    expect(associations).toHaveProperty('versions');
    expect(associations.versions.associationType).toBe('HasMany');
  });
});
```

- [ ] **Step 2: Run test to verify it fails**

Run: `cd src/backend && npx jest src/models/__tests__/Script.associations.test.ts --no-cache 2>&1 | head -30`

Expected: FAIL — `associations.versions` is undefined

- [ ] **Step 3: Add the hasMany association**

In `src/backend/src/models/Script.ts`, inside the `associate()` method, after the existing `Script.belongsToMany(Tag, ...)` line, add:

```
    Script.hasMany(ScriptVersion, { foreignKey: 'scriptId', as: 'versions' });
```

The full associate method becomes:

```
  static associate() {
    Script.belongsTo(User, { foreignKey: 'userId', as: 'user' });
    Script.belongsTo(Category, { foreignKey: 'categoryId', as: 'category' });
    Script.hasOne(ScriptAnalysis, { foreignKey: 'scriptId', as: 'analysis' });
    Script.belongsToMany(Tag, {
      through: 'script_tags',
      foreignKey: 'script_id',
      otherKey: 'tag_id',
      as: 'tags'
    });
    Script.hasMany(ScriptVersion, { foreignKey: 'scriptId', as: 'versions' });
  }
```

You must also add the import at the top of the file:

```
import ScriptVersion from './ScriptVersion';
```

- [ ] **Step 4: Run test to verify it passes**

Run: `cd src/backend && npx jest src/models/__tests__/Script.associations.test.ts --no-cache 2>&1 | head -30`

Expected: PASS

- [ ] **Step 5: Commit**

```
 git add src/backend/src/models/Script.ts src/backend/src/models/__tests__/Script.ass
git commit -m "fix: add missing hasMany(ScriptVersion) association to Script model

The AsyncUploadController.getUploadStatus queried Script with
include: ScriptVersion as 'versions', but no such association existed,
causing eager-loading failures."
```

Task 2: Fix AsyncUploadController `Script.create` — Missing `content`, Ghost `uploadId`

Files: - Modify: `src/backend/src/controllers/AsyncUploadController.ts:291-300` (Script.create call)

Two bugs in the `processNextFile` method: 1. `Script.create` passes `uploadId` — this field doesn't exist on the Script model, Sequelize silently ignores it 2. `Script.create` is missing `content: fileContent` — the `content` column is `allowNull: false`, so this INSERT always fails with a

validation error

- [ ] **Step 1: Write the failing test**

Create file `src/backend/src/controllers/__tests__/AsyncUploadController.test.ts`:

```
/**
 * Tests for AsyncUploadController bug fixes.
 * These tests verify that Script.create receives the correct fields.
 */
import Script from '../../models/Script';
import ScriptVersion from '../../models/ScriptVersion';

// We test the data shape, not the full controller flow (that needs HTTP + multer)
describe('AsyncUploadController Script.create contract', () => {
  it('Script model requires content field (allowNull: false)', () => {
    // Verify the model definition enforces content
    const attributes = Script.getAttributes?.() || (Script as any).rawAttributes;
    expect(attributes.content).toBeDefined();
    expect(attributes.content.allowNull).toBe(false);
  });

  it('Script model does NOT have an uploadId field', () => {
    const attributes = Script.getAttributes?.() || (Script as any).rawAttributes;
    expect(attributes.uploadId).toBeUndefined();
  });

  it('ScriptVersion model uses changelog field, not changes', () => {
    const attributes = ScriptVersion.getAttributes?.() || (ScriptVersion as any).rawA
    expect(attributes.changelog).toBeDefined();
    expect(attributes.changes).toBeUndefined();
  });
});
```

- [ ] **Step 2: Run test to verify it passes (these tests document the model contract)**

Run: `cd src/backend && npx jest src/controllers/__tests__/AsyncUploadController.test.ts --no-cache 2>&1 | head -30`

Expected: PASS — these tests prove the model contract that the controller violates

- [ ] **Step 3: Fix the Script.create call and ScriptVersion.create call**

In `src/backend/src/controllers/AsyncUploadController.ts`, replace lines 291-309:

Old code (lines 291-309):

```
// Create script record with authenticated user's ID
const script = await Script.create({
  title: originalFilename,
  description,
  uploadId,
  userId, // Use the authenticated user's ID from queue context
  categoryId,
  isPublic: false,
  executionCount: 0
}, { transaction });

// Create script version with authenticated user's ID
await ScriptVersion.create({
  scriptId: script.id,
  version: 1,
  content: fileContent,
  changes: 'Initial version',
  userId // Use the authenticated user's ID from queue context
}, { transaction });
```

New code:

```
// Create script record with authenticated user's ID
const script = await Script.create({
  title: originalFilename,
  description,
  content: fileContent,
  userId,
  categoryId,
  isPublic: false,
  executionCount: 0
}, { transaction });

// Create script version with authenticated user's ID
await ScriptVersion.create({
  scriptId: script.id,
  version: 1,
  content: fileContent,
  changelog: 'Initial version',
  userId
}, { transaction });
```

Changes: - Removed `uploadId` (doesn't exist on model) - Added `content: fileContent` (required field) - Changed `changes` to `changelog` (correct model field name)

- [ ] **Step 4: Run tests to verify**

Run: `cd src/backend && npx jest src/controllers/__tests__/AsyncUploadController.test.ts --no-cache 2>&1 | head -30`

Expected: PASS

- [ ] **Step 5: Commit**

```
 git add src/backend/src/controllers/AsyncUploadController.ts src/backend/src/control
git commit -m "fix: AsyncUploadController Script.create missing content, wrong field

Three bugs fixed:
1. Added missing content field to Script.create (allowNull: false)
2. Removed ghost uploadId field (not in Script model)
3. Changed 'changes' to 'changelog' in ScriptVersion.create"
```

Task 3: Fix `getUploadStatus` — Broken Status Tracking

Files: - Modify: `src/backend/src/controllers/AsyncUploadController.ts:191-210` (getUploadStatus method)

The `getUploadStatus` method queries `Script.findOne({ where: { uploadId } })` but `uploadId` doesn't exist on the Script model. This means the "completed" status check always returns null, so uploads that succeed still show as "not_found".

Strategy: Track completed uploads using the existing `cache` service (Redis + in-memory fallback). This gives durability across restarts and shared state across multiple backend instances. Store `upload:status:{uploadId} → { scriptId, title, description, versions }` with a 1-hour TTL.

- [ ] **Step 1: Add cache import**

In `src/backend/src/controllers/AsyncUploadController.ts`, add import at the top:

```
import cache from '../services/cacheService';
```

- [ ] **Step 2: Add helper methods for upload status persistence**

Add these private methods to the AsyncUploadController class:

```
private static readonly UPLOAD_STATUS_TTL = 3600; // 1 hour in seconds

private storeCompletedUpload(uploadId: string, data: {
  scriptId: number; title: string; description: string; versions: number;
}): void {
  cache.set(`upload:status:${uploadId}`, data, AsyncUploadController.UPLOAD_STATUS_
}

private getCompletedUpload(uploadId: string): {
  scriptId: number; title: string; description: string; versions: number;
} | null {
  return cache.get(`upload:status:${uploadId}`);
}
```

- [ ] **Step 3: Store completed upload info after successful processing**

In the `processNextFile` method, after the line `logger.info(\`Successfully processed file: ${uploadId}`);` (line 314), add:

```
        // Track completed upload for status queries (Redis-backed, 1hr TTL)
        this.storeCompletedUpload(uploadId, {
          scriptId: script.id,
          title: script.title,
          description: description || '',
          versions: 1
        });
```

- [ ] **Step 4: Rewrite the getUploadStatus completed check**

Replace the database query block in `getUploadStatus` (lines 191-210):

Old code:

```
      // Check if script was created in database
      const script = await Script.findOne({
        where: { uploadId },
        include: [{ model: ScriptVersion, as: 'versions' }]
      });

      if (script) {
        res.json({
          success: true,
          status: 'completed',
          message: 'File processing completed successfully',
          scriptId: script.id,
          scriptDetails: {
            title: script.title,
            description: script.description,
            versions: (script as any).versions?.length || 1
          }
        });
        return;
      }
```

New code:

```
      // Check if upload completed successfully (Redis-backed, shared across instanc
      const completed = this.getCompletedUpload(uploadId);
      if (completed) {
        res.json({
          success: true,
          status: 'completed',
          message: 'File processing completed successfully',
          scriptId: completed.scriptId,
          scriptDetails: {
            title: completed.title,
            description: completed.description,
            versions: completed.versions
          }
        });
        return;
      }
```

- [ ] **Step 4: Run typecheck**

Run: `cd src/backend && npx tsc --noEmit 2>&1 | head -30`

Expected: No errors related to AsyncUploadController

- [ ] **Step 5: Commit**

```
 git add src/backend/src/controllers/AsyncUploadController.ts
git commit -m "fix: getUploadStatus was querying non-existent uploadId column

Replaced broken Script.findOne({ where: { uploadId } }) with
Redis-backed cache (1hr TTL), durable across restarts and shared
across multiple backend instances."
```

Task 4: Fix Race Condition in Hash Deduplication

Files: - Modify: `src/backend/src/controllers/script/export.ts:506-519` (hash check) - Create: `src/db/migrations/add_unique_file_hash.sql`

The hash deduplication check happens outside the transaction. Two concurrent uploads with identical content can both pass the check. Fix: move check inside transaction + add UNIQUE constraint.

- [ ] **Step 1: Create the migration file**

Create file `src/db/migrations/add_unique_file_hash.sql`:

```
-- PREREQUISITE: Task 5 migration (upgrade_file_hash_sha256.sql) must have run first
-- to rehash all MD5 values to SHA-256.
--
-- Step 1: Audit any remaining duplicate hashes into a temp table for review.
-- This preserves a record of what was merged, so no data is silently lost.
CREATE TABLE IF NOT EXISTS _duplicate_hash_audit AS
  SELECT a.id AS removed_id, b.id AS kept_id, a.file_hash, a.title, a.created_at
  FROM scripts a
  JOIN scripts b ON a.file_hash = b.file_hash AND a.id < b.id
  WHERE a.file_hash IS NOT NULL;

-- Step 2: Reassign dependent records (versions, tags, analyses) from duplicates to t
-- This prevents orphaned foreign key references.
UPDATE script_versions SET script_id = d.kept_id
  FROM _duplicate_hash_audit d WHERE script_versions.script_id = d.removed_id;

UPDATE script_tags SET script_id = d.kept_id
  FROM _duplicate_hash_audit d WHERE script_tags.script_id = d.removed_id
  ON CONFLICT DO NOTHING;  -- skip if the kept script already has that tag

UPDATE analysis_results SET script_id = d.kept_id
  FROM _duplicate_hash_audit d WHERE analysis_results.script_id = d.removed_id;

-- Step 3: Remove the duplicate rows (now safe — dependents have been reassigned).
DELETE FROM scripts WHERE id IN (SELECT removed_id FROM _duplicate_hash_audit);

-- Step 4: Add UNIQUE constraint on file_hash to prevent future duplicates.
-- Null values are allowed (scripts without hashes) — PostgreSQL UNIQUE permits multi
ALTER TABLE scripts ADD CONSTRAINT uq_scripts_file_hash UNIQUE (file_hash);

-- Note: The _duplicate_hash_audit table is kept for post-migration review.
-- Drop it manually after verifying the migration was successful:
--   DROP TABLE IF EXISTS _duplicate_hash_audit;
```

> **Important:** Before running this migration in production, verify the dependent table names match your schema (the above uses `script_versions`, `script_tags`, `analysis_results`). Check with: `SELECT table_name FROM information_schema.tables WHERE table_name LIKE 'script%' OR table_name LIKE 'analysis%';`

- [ ] **Step 2: Update Script model index to be unique**

In `src/backend/src/models/Script.ts`, change the `file_hash` index from non-unique to unique:

Old:

```
        { fields: ['file_hash'], name: 'idx_scripts_file_hash' },
```

New:

```
        { unique: true, fields: ['file_hash'], name: 'idx_scripts_file_hash' },
```

- [ ] **Step 3: Move hash check inside the transaction in export.ts**

In `src/backend/src/controllers/script/export.ts`, the hash check at lines 512-519 currently happens after the transaction starts but reads outside of it. Update the `checkFileExists` call to pass the transaction:

Replace lines 512-519:

Old code:

```
    // Check if a file with the same hash already exists
    const existingScriptId = await checkFileExists(fileHash, sequelize);
    if (existingScriptId) {
      if (transaction) await transaction.rollback();
      return res.status(409).json({
        error: 'duplicate_file',
        message: 'A script with identical content already exists',
        existingScriptId
      });
    }
```

New code:

```
    // Check if a file with the same hash already exists (inside transaction for ato
    const existingScriptId = await checkFileExists(fileHash, sequelize, transaction);
    if (existingScriptId) {
      await transaction.rollback();
      return res.status(409).json({
        error: 'duplicate_file',
        message: 'A script with identical content already exists',
        existingScriptId
      });
    }
```

- [ ] **Step 4: Update checkFileExists to accept an optional transaction**

In `src/backend/src/utils/fileIntegrity.ts`, update the `checkFileExists` function signature and implementation:

Old code:

```
export const checkFileExists = async (fileHash: string, sequelize: Sequelize): Promi
  try {
    const [result] = await sequelize.query(
      `SELECT id FROM scripts WHERE file_hash = :fileHash LIMIT 1`,
      {
        replacements: { fileHash },
        type: 'SELECT',
        raw: true
      }
    );
```

New code:

```
export const checkFileExists = async (
  fileHash: string,
  sequelize: Sequelize,
  transaction?: import('sequelize').Transaction
): Promise<number | null> => {
  try {
    const [result] = await sequelize.query(
      `SELECT id FROM scripts WHERE file_hash = :fileHash LIMIT 1 FOR UPDATE`,
      {
        replacements: { fileHash },
        type: 'SELECT',
        raw: true,
        ...(transaction ? { transaction } : {})
      }
    );
```

The `FOR UPDATE` clause locks the matching row within the transaction, preventing a second concurrent upload from reading a stale state while the first is still inserting. Note: `FOR UPDATE` only locks rows that match the WHERE clause — when no matching row exists (new hash), the UNIQUE constraint (Step 1) is the primary defense against concurrent duplicate inserts.

- [ ] **Step 5: Add SequelizeUniqueConstraintError handling as a safety net**

In `src/backend/src/controllers/script/export.ts`, the catch block at line 716 already handles `SequelizeUniqueConstraintError` but returns a generic "title already exists" message. Update it to also handle hash conflicts:

Old code (line 716-719):

```
    if (err.name === 'SequelizeUniqueConstraintError') {
      return res.status(409).json({
        error: 'unique_constraint_error',
        message: 'A script with this title already exists'
      });
    }
```

New code:

```
    if (err.name === 'SequelizeUniqueConstraintError') {
      const fields = (err as any).fields || {};
      if (fields.file_hash) {
        return res.status(409).json({
          error: 'duplicate_file',
          message: 'A script with identical content already exists'
        });
      }
      return res.status(409).json({
        error: 'unique_constraint_error',
        message: 'A script with this title already exists'
      });
    }
```

- [ ] **Step 6: Run typecheck**

Run: `cd src/backend && npx tsc --noEmit 2>&1 | head -30`

Expected: No errors

- [ ] **Step 7: Commit**

```
 git add src/backend/src/models/Script.ts src/backend/src/controllers/script/export.t
git commit -m "fix: race condition in hash deduplication during concurrent uploads

Moved hash check inside transaction with FOR UPDATE lock.
Added UNIQUE constraint on file_hash column as DB-level safety net.
Improved error message for hash-based duplicate detection."
```

Task 5: Upgrade Hash Algorithm from MD5 to SHA-256

Files: - Modify: `src/backend/src/utils/fileIntegrity.ts` - Modify: `src/backend/src/utils/fileIntegrity.ts` - Create: `src/db/migrations/upgrade_file_hash_sha256.sql`

CLAUDE.md documents SHA-256 for file integrity, but the code uses MD5 which is cryptographically broken. The `fileHash` column is already `STRING(64)` — enough for SHA-256 hex output (64 chars), so no column change is needed. But we need a migration to rehash existing MD5 values (32 chars) to SHA-256 (64 chars).

- [ ] **Step 1: Create migration to rehash existing scripts**

Create file `src/db/migrations/upgrade_file_hash_sha256.sql`:

```
-- Upgrade file hashes from MD5 (32 hex chars) to SHA-256 (64 hex chars).
-- Recompute in-place to avoid a dedup gap where uploads could bypass duplicate detec
-- The application now uses SHA-256 for all new uploads.
CREATE EXTENSION IF NOT EXISTS pgcrypto;
UPDATE scripts
  SET file_hash = encode(digest(content::text, 'sha256'), 'hex')
  WHERE file_hash IS NOT NULL AND length(file_hash) = 32;
```

- [ ] **Step 2: Update fileIntegrity.ts to use SHA-256**

In `src/backend/src/utils/fileIntegrity.ts`:

Replace `calculateBufferMD5`:

Old:

```
export const calculateBufferMD5 = (buffer: Buffer): string => {
  return crypto.createHash('md5').update(buffer).digest('hex');
};
```

New:

```
export const calculateBufferSHA256 = (buffer: Buffer): string => {
  return crypto.createHash('sha256').update(buffer).digest('hex');
};

/** @deprecated Use calculateBufferSHA256 */
export const calculateBufferMD5 = calculateBufferSHA256;
```

Replace `calculateStringMD5`:

Old:

```
export const calculateStringMD5 = (content: string): string => {
  return crypto.createHash('md5').update(content).digest('hex');
};
```

New:

```
export const calculateStringSHA256 = (content: string): string => {
  return crypto.createHash('sha256').update(content).digest('hex');
};

/** @deprecated Use calculateStringSHA256 */
export const calculateStringMD5 = calculateStringSHA256;
```

Update `batchUpdateFileHashes` to use SHA-256:

Old:

```
      `UPDATE scripts SET file_hash = md5(content::text) WHERE file_hash IS NULL RET
```

New:

```
      `UPDATE scripts SET file_hash = encode(digest(content::text, 'sha256'), 'hex')
```

Note: This uses PostgreSQL's `pgcrypto` extension. The project already has `pgvector` so extensions are available. If `pgcrypto` isn't installed, the migration should add it.

- [ ] **Step 3: Verify pgcrypto is included in migration**

The migration file from Step 1 already includes `CREATE EXTENSION IF NOT EXISTS pgcrypto` and recomputes hashes in-place. Verify the final file contains both statements. If deploying to an environment where migrations run separately from app startup, ensure this migration runs before any new uploads occur.

- [ ] **Step 4: Run typecheck**

Run: `cd src/backend && npx tsc --noEmit 2>&1 | head -30`

Expected: No errors (deprecated alias keeps all existing imports working)

- [ ] **Step 5: Commit**

```
 git add src/backend/src/utils/fileIntegrity.ts src/db/migrations/upgrade_file_hash_s
git commit -m "fix: upgrade file hash from MD5 to SHA-256

MD5 is cryptographically broken. SHA-256 matches the documented
algorithm in CLAUDE.md. Existing MD5 hashes recomputed in-place as SHA-256.
Old function names kept as deprecated aliases for backward compatibility."
```

Task 6: Add Hash Deduplication to AsyncUploadController

Files: - Modify: `src/backend/src/controllers/AsyncUploadController.ts:260-300`

The `AsyncUploadController.processNextFile` does NOT check for duplicate hashes before creating a script. Only `export.ts` (the synchronous upload) does dedup. This means async bulk uploads can create duplicates.

- [ ] **Step 1: Add hash check to processNextFile**

In `src/backend/src/controllers/AsyncUploadController.ts`, add imports at the top (after existing imports):

```
import { calculateBufferSHA256 as calculateHash, checkFileExists } from '../utils/fi
```

Then in `processNextFile`, after reading the file content (line 260: `const fileContent = await readFileAsync(filePath, 'utf8');`), compute the hash before the transaction, but perform the dedup check **inside** the transaction (consistent with Task 4's race-condition fix):

```
      // Calculate hash before transaction
      const fileHash = calculateHash(Buffer.from(fileContent, 'utf8'));
```

Then **inside** the transaction block (after `const transaction = await sequelize.transaction();`), add the dedup check:

```
        // Check for duplicates inside transaction (prevents race condition — see
        const existingScriptId = await checkFileExists(fileHash, sequelize, transac
        if (existingScriptId) {
          await transaction.rollback();
          logger.info(`Duplicate detected for upload ${uploadId}, existing script I
          this.storeCompletedUpload(uploadId, {
            scriptId: existingScriptId,
            title: `(duplicate of ${existingScriptId})`,
            description: '',
            versions: 1
          });
          await unlinkAsync(filePath);
          return;
        }
```

Also add `fileHash` to the `Script.create` call:

```
        const script = await Script.create({
          title: originalFilename,
          description,
          content: fileContent,
          fileHash,
          userId,
          categoryId,
          isPublic: false,
          executionCount: 0
        }, { transaction });
```

> **Note:** The UNIQUE constraint on `file_hash` (Task 4) is the primary defense against concurrent duplicate inserts. The `FOR UPDATE` lock in `checkFileExists` helps only when a matching row already exists — when no row matches, `FOR UPDATE` has nothing to lock. Two concurrent transactions can both see zero rows and proceed to INSERT, at which point the UNIQUE constraint catches the second one.

- [ ] **Step 2: Run typecheck**

Run: `cd src/backend && npx tsc --noEmit 2>&1 | head -30`

Expected: No errors

- [ ] **Step 3: Commit**

```
 git add src/backend/src/controllers/AsyncUploadController.ts
git commit -m "fix: add hash dedup to AsyncUploadController

Async bulk uploads were not checking for duplicate content,
allowing identical scripts to be stored multiple times.
Now computes SHA-256 hash and checks before insert."
```

Self-Review Checklist

Spec coverage: - [x] Missing `content` field in Script.create → Task 2 - [x] Ghost `uploadId` field → Task 2, Task 3 - [x] Wrong field name `changes` → Task 2 - [x] Missing Script-ScriptVersion association → Task 1 - [x] Race condition in hash dedup → Task 4 - [x] MD5 vs SHA-256 mismatch → Task 5 - [x] No dedup in async upload path → Task 6 - [x] Broken upload status tracking → Task 3

Placeholder scan: No TBDs, TODOs, or "similar to Task N" references found.

Type consistency: - `calculateBufferMD5` aliased to `calculateBufferSHA256` — all imports continue to work (Task 6 uses `calculateBufferSHA256` directly) - `checkFileExists` gains optional `transaction` param — backward compatible - `changelog` field name used consistently after Task 2 fix

---

Generated 2026-04-29 14:08 UTC